

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

DHRUVA ACHARYA N (1BM24CS093)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019
February to June 2026
CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Dhruva Acharya N (**1BM24CS093**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Mayanka Gupta
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	6
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	11
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	14
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	17
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	19
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	23
9	Implement Fractional Knapsack using Greedy technique.	26
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	28
11	Implement "N-Queens Problem" using Backtracking.	31
12	Leetcode Problems	34

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program 1:

Experiment Title: Write program to obtain the Topological ordering of vertices in a given digraph.

Aim:

To implement and demonstrate Topological Sorting of a directed acyclic graph (DAG) using Kahn's algorithm based on in-degree computation.

Program (C):

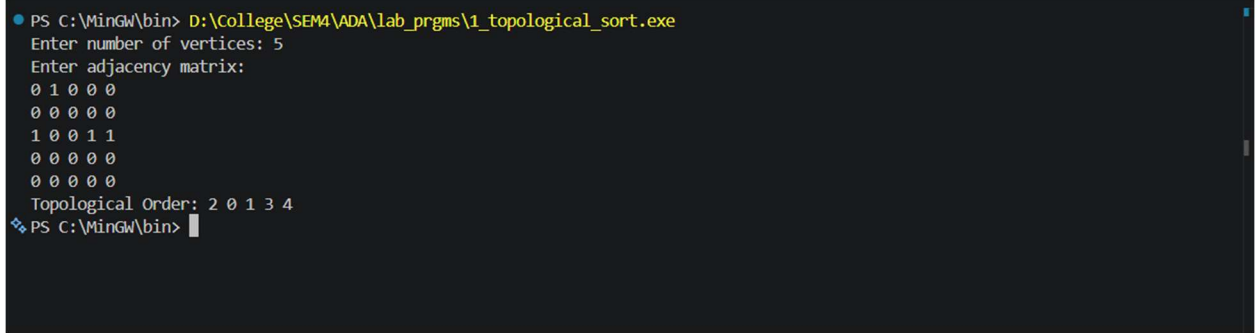
```
#include<stdio.h>
int a[20][20],in[20],n;
void topo()
{
    int i,j,k;
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            in[j]+=a[i][j];
    printf("Topological Order: ");
    for(k=0;k<n;k++)
    {
        for(i=0;i<n;i++)
        {
            if(in[i]==0)
            {
                printf("%d ",i);
                in[i]=-1;
                for(j=0;j<n;j++)
                    if(a[i][j]) in[j]--;
                break;
            }
        }
    }
}
```

```

    }
}
}
int main()
{
    int i,j;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    printf("Enter adjacency matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&a[i][j]);
    topo();
}

```

Output:



```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\1_topological_sort.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 1 0 0 0
0 0 0 0 0
1 0 0 1 1
0 0 0 0 0
0 0 0 0 0
Topological Order: 2 0 1 3 4
PS C:\MinGW\bin>

```

Lab Program 2:

Experiment Title: Implement Johnson Trotter algorithm to generate permutations.

Aim:

To generate all permutations of n elements using the Johnson-Trotter algorithm, which produces permutations by successively swapping adjacent elements.

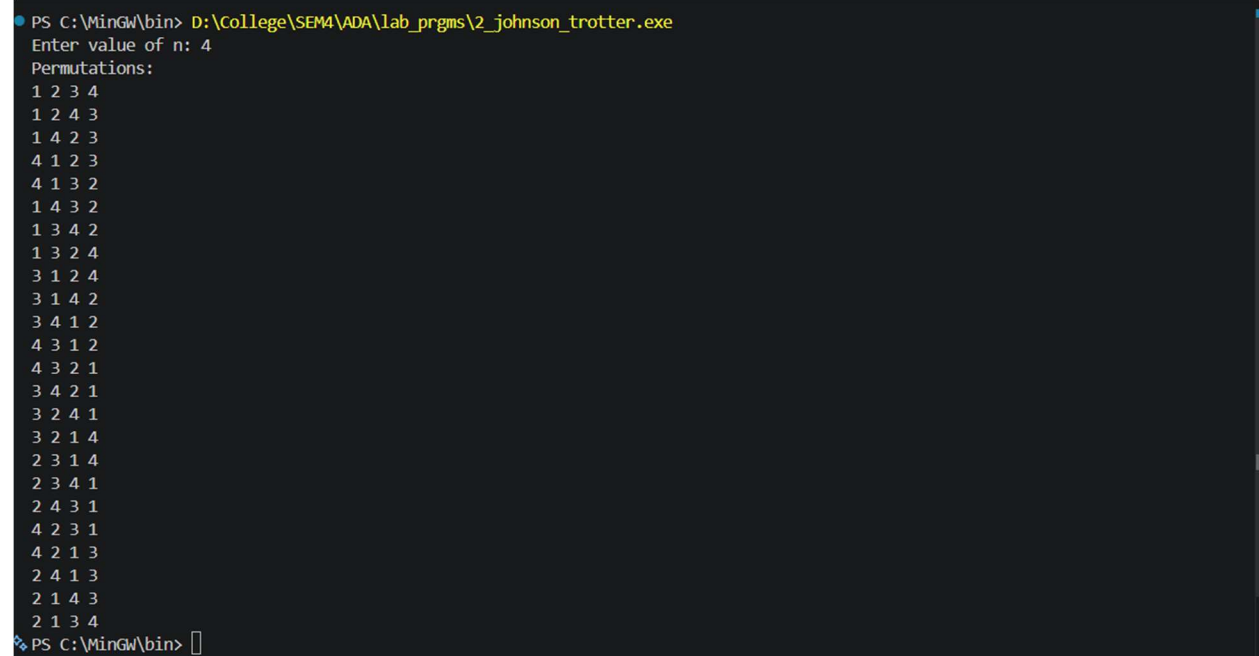
Program (C):

```
#include<stdio.h>

int main()
{
    int n,i,j,m,p,t;
    printf("Enter value of n: ");
    scanf("%d",&n);
    int a[n],d[n];
    for(i=0;i<n;i++) { a[i]=i+1; d[i]=-1; }
    printf("Permutations:\n");
    while(1)
    {
        for(i=0;i<n;i++) printf("%d ",a[i]);
        printf("\n");
        m=-1;
        for(i=0;i<n;i++)
        {
            if(d[a[i]-1]==-1 && i>0 && a[i]>a[i-1] && a[i]>m) m=a[i];
            if(d[a[i]-1]==1 && i<n-1 && a[i]>a[i+1] && a[i]>m) m=a[i];
        }
        if(m==-1) break;
        for(i=0;i<n;i++) if(a[i]==m) p=i;
        j=p+d[m-1];
        t=a[p]; a[p]=a[j]; a[j]=t;
    }
}
```

```
        for(i=m;i<n;i++) d[i]=-d[i];  
    }  
}
```

Output:



```
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\2_johnson_trotter.exe  
Enter value of n: 4  
Permutations:  
1 2 3 4  
1 2 4 3  
1 4 2 3  
4 1 2 3  
4 1 3 2  
1 4 3 2  
1 3 4 2  
1 3 2 4  
3 1 2 4  
3 1 4 2  
3 4 1 2  
4 3 1 2  
4 3 2 1  
3 4 2 1  
3 2 4 1  
3 2 1 4  
2 3 1 4  
2 3 4 1  
2 4 3 1  
4 2 3 1  
4 2 1 3  
2 4 1 3  
2 1 4 3  
2 1 3 4  
PS C:\MinGW\bin>
```

Lab Program 3:

Experiment Title: Sort a given set of N integer elements using Merge Sort technique and compute its time taken.

Aim:

To sort N integer elements using Merge Sort (Divide-and-Conquer) and measure execution time for different values of N to study the time complexity.

Program (C):

```
#include<stdio.h>
#include<time.h>
void merge(int a[],int l,int m,int r)
{
    int i=l,j=m+1,k=0,b[100000];
    while(i<=m && j<=r){
        b[k++]=(a[i]<a[j])?a[i++]:a[j++];
    }
    while(i<=m) b[k++]=a[i++];
    while(j<=r) b[k++]=a[j++];
    for(i=l,k=0;i<=r;i++,k++) a[i]=b[k];
}
void mergesort(int a[],int l,int r)
{
    if(l<r)
    {
        int m=(l+r)/2;
        mergesort(a,l,m);
        mergesort(a,m+1,r);
        merge(a,l,m,r);
    }
}
```

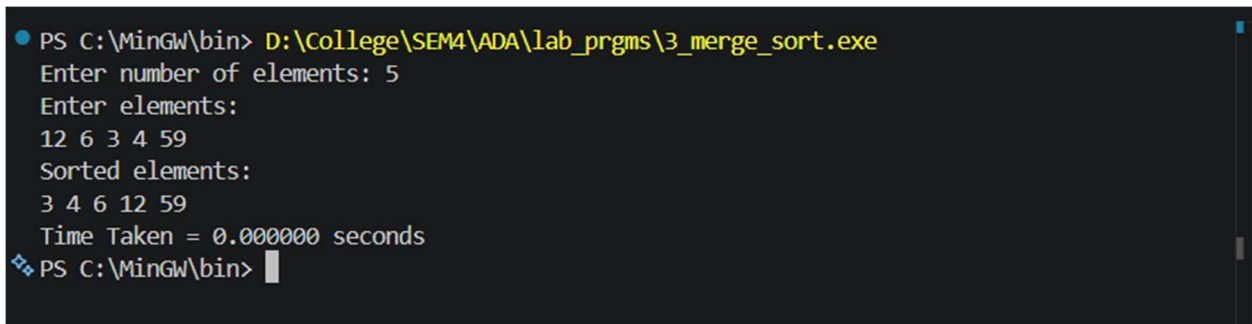


```

int main()
{
    int n,i,a[100000];
    clock_t s,e;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    s=clock();
    mergesort(a,0,n-1);
    e=clock();
    printf("Sorted elements:\n");
    for(i=0;i<n;i++) printf("%d ",a[i]);
    printf("\nTime Taken = %lf seconds",(double)(e-s)/CLOCKS_PER_SEC);
}

```

Output:



```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\3_merge_sort.exe
Enter number of elements: 5
Enter elements:
12 6 3 4 59
Sorted elements:
3 4 6 12 59
Time Taken = 0.000000 seconds
PS C:\MinGW\bin>

```

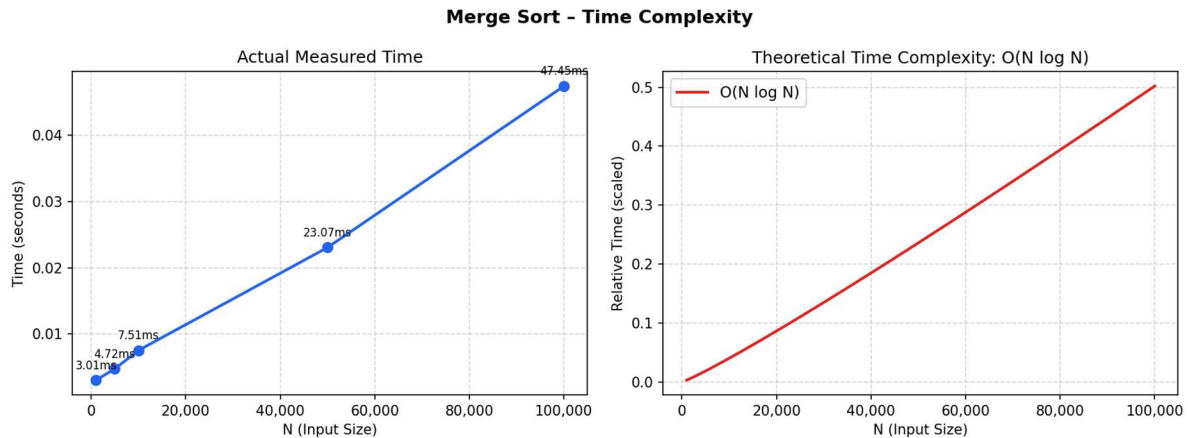
Time Complexity Analysis:

The program was executed for various input sizes (N) with random data. The recorded execution times are tabulated below:

N (Input Size)	Time Taken (ms)
1,000	3.012
5,000	4.720
10,000	7.513
50,000	23.067
100,000	47.453

Time Complexity: $O(N \log N)$

Time Complexity Graphs:



Left: Actual measured execution time | Right: Theoretical $O(N \log N)$ curve

Lab Program 4:

Experiment Title: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Aim:

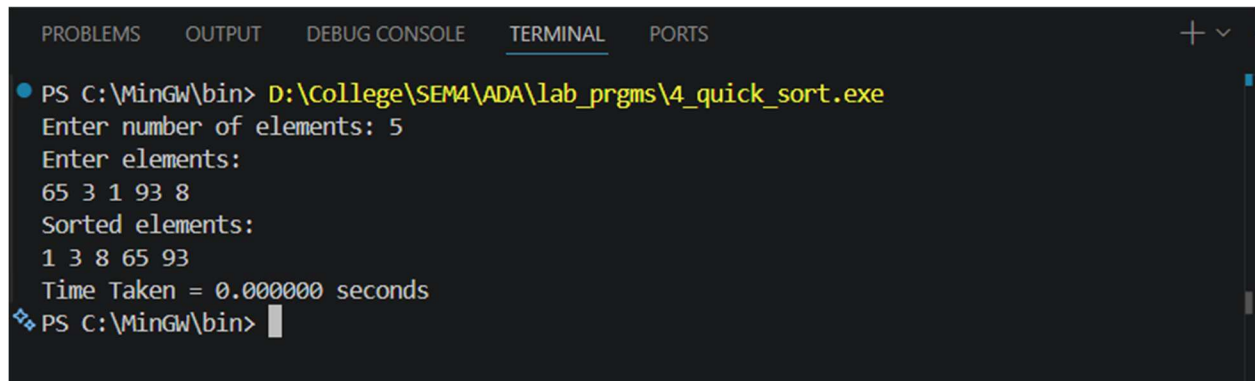
To sort N integer elements using Quick Sort (Divide-and-Conquer with partitioning) and measure execution time for different input sizes.

Program (C):

```
#include<stdio.h>
#include<time.h>
void quicksort(int a[],int l,int r)
{
    if(l<r)
    {
        int i=l,j=r,p=a[l],t;
        while(i<j)
        {
            while(a[i]<=p && i<r) i++;
            while(a[j]>p) j--;
            if(i<j){ t=a[i];a[i]=a[j];a[j]=t; }
        }
        a[l]=a[j]; a[j]=p;
        quicksort(a,l,j-1);
        quicksort(a,j+1,r);
    }
}
int main()
{
    int n,i,a[100000];
    clock_t s,e;
```

```
printf("Enter number of elements: ");
scanf("%d",&n);
printf("Enter elements:\n");
for(i=0;i<n;i++) scanf("%d",&a[i]);
s=clock();
quicksort(a,0,n-1);
e=clock();
printf("Sorted elements:\n");
for(i=0;i<n;i++) printf("%d ",a[i]);
printf("\nTime Taken = %lf seconds",(double)(e-s)/CLOCKS_PER_SEC);
}
```

Output:

A screenshot of a terminal window with a dark background. The terminal shows the execution of a program named '4_quick_sort.exe'. The user enters '5' for the number of elements and '65 3 1 93 8' for the elements. The program outputs the sorted elements '1 3 8 65 93' and the time taken '0.000000 seconds'. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The prompt is 'PS C:\MinGW\bin>'.

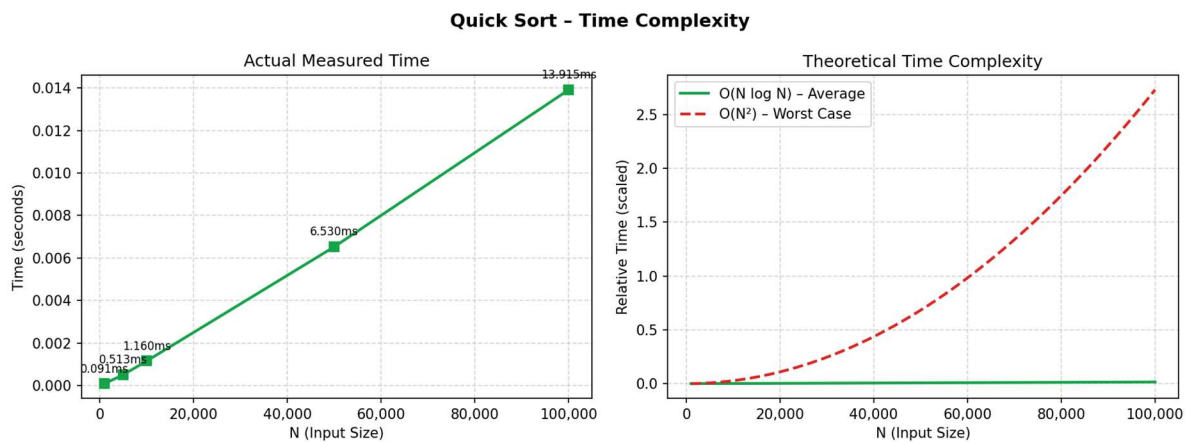
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\4_quick_sort.exe
Enter number of elements: 5
Enter elements:
65 3 1 93 8
Sorted elements:
1 3 8 65 93
Time Taken = 0.000000 seconds
PS C:\MinGW\bin>
```

Time Complexity Analysis:

N (Input Size)	Time Taken (ms)
1,000	0.091
5,000	0.513
10,000	1.160
50,000	6.530
100,000	13.915

Time Complexity: $O(N \log N)$ average, $O(N^2)$ worst case

Time Complexity Graphs:



Left: Actual measured execution time | Right: Theoretical $O(N \log N)$ average and $O(N^2)$ worst case curves

Lab Program 5:

Experiment Title: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Aim:

To sort N integer elements using Heap Sort and analyze the execution time for varying input sizes, demonstrating $O(N \log N)$ time complexity.

Program (C):

```
#include<stdio.h>
#include<time.h>
void heapify(int a[],int n,int i)
{
    int l=2*i+1, r=2*i+2, max=i, t;
    if(l<n && a[l]>a[max]) max=l;
    if(r<n && a[r]>a[max]) max=r;
    if(max!=i)
    {
        t=a[i]; a[i]=a[max]; a[max]=t;
        heapify(a,n,max);
    }
}
int main()
{
    int n,i,t,a[100000];
    clock_t s,e;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    s=clock();
```

```

for(i=n/2-1;i>=0;i--) heapify(a,n,i);
for(i=n-1;i>0;i--)
{
    t=a[0]; a[0]=a[i]; a[i]=t;
    heapify(a,i,0);
}
e=clock();
printf("Sorted elements:\n");
for(i=0;i<n;i++) printf("%d ",a[i]);
printf("\nTime Taken = %lf seconds",(double)(e-s)/CLOCKS_PER_SEC);
}

```

Output:

```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\5_heap_sort.exe
Enter number of elements: 7
Enter elements:
3 1 76 0 37 8 2
Sorted elements:
0 1 2 3 8 37 76
Time Taken = 0.000000 seconds
PS C:\MinGW\bin>

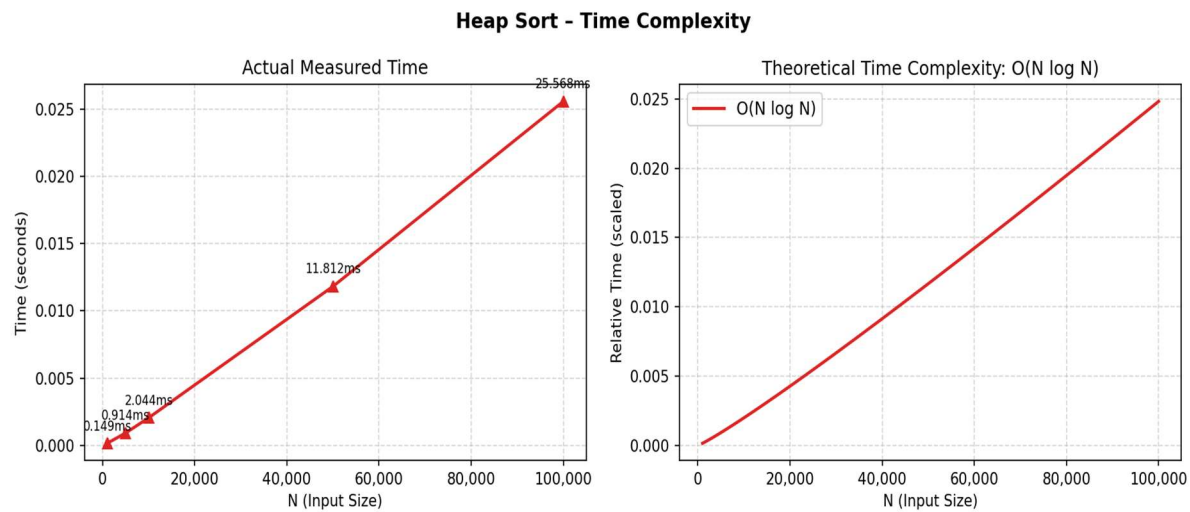
```

Time Complexity Analysis:

N (Input Size)	Time Taken (ms)
1,000	0.149
5,000	0.914
10,000	2.044
50,000	11.812
100,000	25.568

Time Complexity: $O(N \log N)$

Time Complexity Graphs:



Left: Actual measured execution time | Right: Theoretical $O(N \log N)$ curve

Comparative Time Complexity Graph – Sorting Algorithms

The graph below shows the execution time (in seconds) for Merge Sort, Quick Sort, and Heap Sort for input sizes ranging from 1,000 to 1,00,000 elements using randomly generated numbers as input,

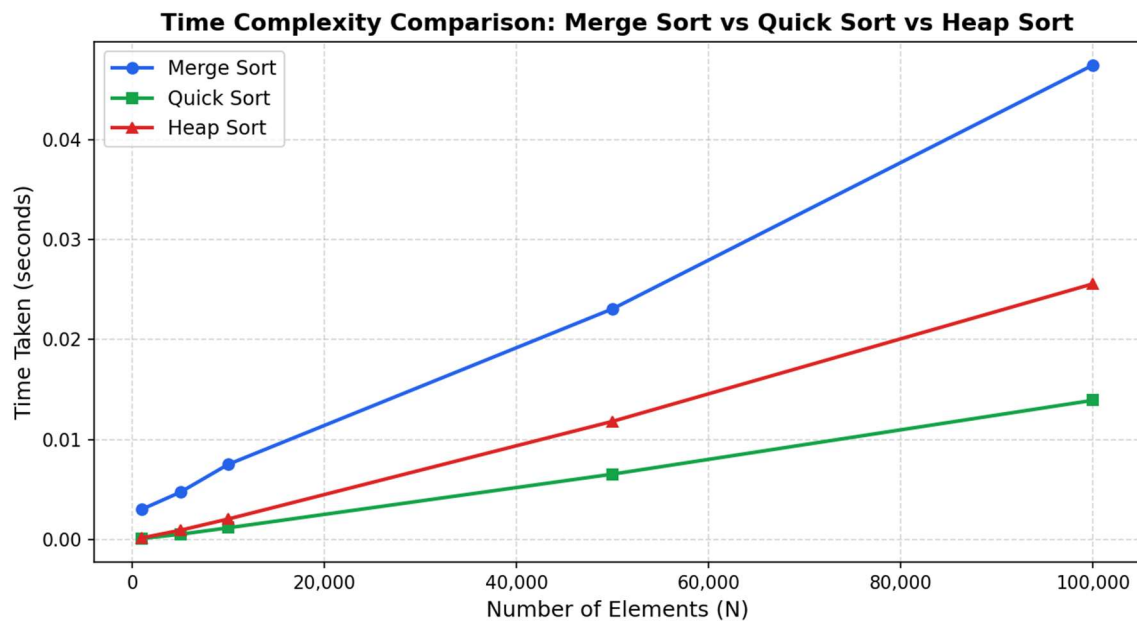


Figure: Time vs N graph for Merge Sort, Quick Sort, and Heap Sort

Lab Program 6:

Experiment Title: Implement 0/1 Knapsack problem using dynamic programming.

Aim:

To solve the 0/1 Knapsack Problem using Dynamic Programming, maximizing the total profit without exceeding the knapsack capacity. Time Complexity: $O(n*W)$.

Program (C):

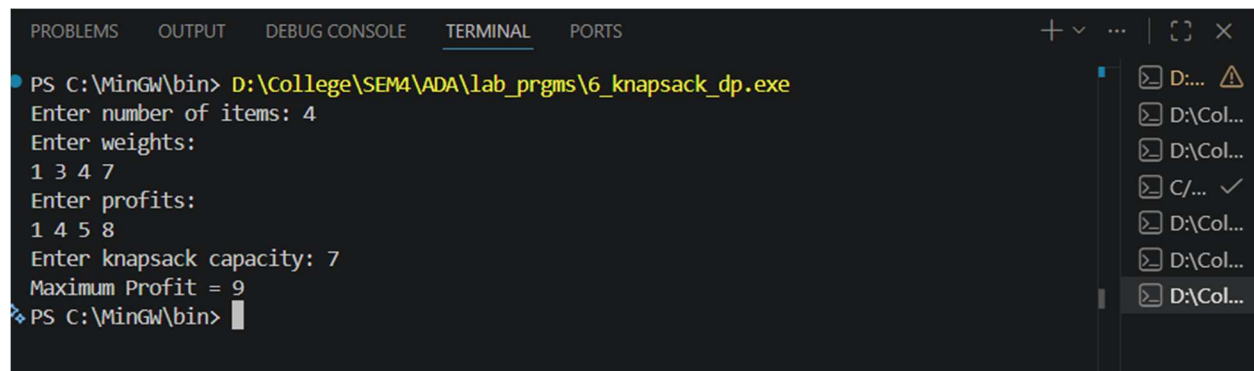
```
#include<stdio.h>

int max(int a,int b) { return (a>b)?a:b; }

int main()
{
    int n,w,i,j;
    printf("Enter number of items: ");
    scanf("%d",&n);
    int wt[n],p[n];
    printf("Enter weights:\n");
    for(i=0;i<n;i++) scanf("%d",&wt[i]);
    printf("Enter profits:\n");
    for(i=0;i<n;i++) scanf("%d",&p[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d",&w);
    int k[n+1][w+1];
    for(i=0;i<=n;i++)
    {
        for(j=0;j<=w;j++)
        {
            if(i==0 || j==0) k[i][j]=0;
            else if(wt[i-1]<=j)
                k[i][j]=max(p[i-1]+k[i-1][j-wt[i-1]], k[i-1][j]);
            else
```

```
        k[i][j]=k[i-1][j];  
    }  
}  
printf("Maximum Profit = %d",k[n][w]);  
}
```

Output:



The screenshot shows a terminal window with the following text:

```
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\6_knapsack_dp.exe  
Enter number of items: 4  
Enter weights:  
1 3 4 7  
Enter profits:  
1 4 5 8  
Enter knapsack capacity: 7  
Maximum Profit = 9  
PS C:\MinGW\bin>
```

The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. On the right side, there is a file explorer showing a list of files and folders, including D:\College\SEM4\ADA\lab_prgms\6_knapsack_dp.exe.

Lab Program 7:

Experiment Title: Implement All Pair Shortest paths problem using Floyd's algorithm.

Aim:

To find the shortest paths between all pairs of vertices in a weighted graph using Floyd's Algorithm (Dynamic Programming). Time Complexity: $O(V^3)$.

Program (C):

```
#include<stdio.h>
#define INF 99999
int main(){
    int n,i,j,k;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int d[n][n];
    printf("Enter adjacency matrix (use 99999 for no edge):\n");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&d[i][j]);
    for(k=0;k<n;k++)
        for(i=0;i<n;i++)
            for(j=0;j<n;j++)
                if(d[i][k]+d[k][j]<d[i][j])
                    d[i][j]=d[i][k]+d[k][j];
    printf("Shortest Path Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++)
            printf("%5d ",d[i][j]);
        printf("\n");
    }
}
```

Output:

```
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\7_floyds.exe
Enter number of vertices: 4
Enter adjacency matrix (use 99999 for no edge):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0
Shortest Path Matrix:
    0    5    8    9
99999    0    3    4
99999 99999    0    1
99999 99999 99999    0
PS C:\MinGW\bin>
```

Lab Program 8:

Experiment Title: Find Minimum Cost Spanning Tree using Prim's and Kruskal's algorithms.

8A – Prim's Algorithm

Aim:

To find the Minimum Spanning Tree (MST) of an undirected weighted graph using Prim's greedy algorithm. Time Complexity: $O(V^2)$.

Program (C):

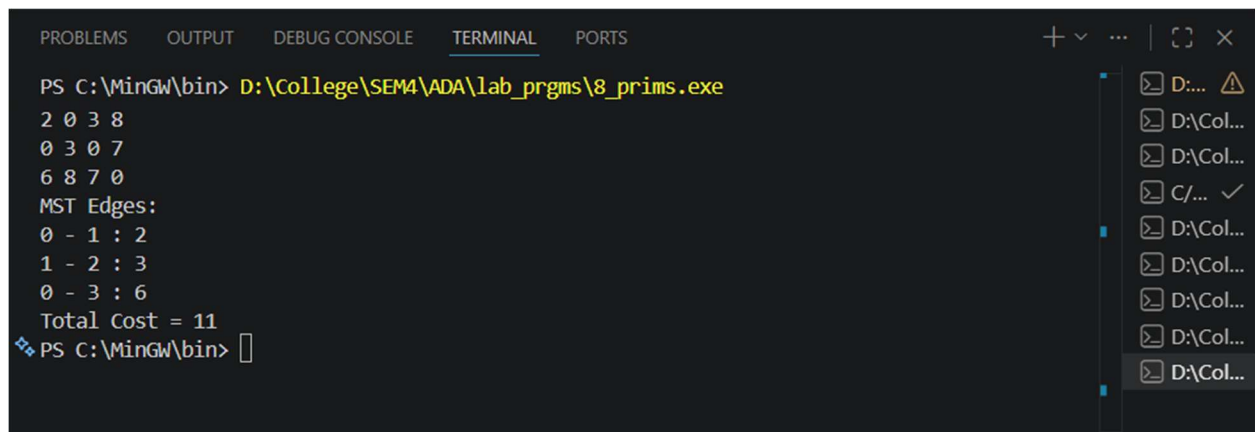
```
#include<stdio.h>
#define INF 99999
int main()
{
    int n,i,j,u,v,min,cost=0;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int g[n][n],vis[n],key[n],par[n];
    printf("Enter adjacency matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&g[i][j]);
    for(i=0;i<n;i++){
        vis[i]=0; key[i]=INF; par[i]=-1;
    }
    key[0]=0;
    for(i=0;i<n-1;i++){
        min=INF; u=-1;
        for(j=0;j<n;j++)
            if(!vis[j] && key[j]<min){ min=key[j]; u=j; }
        vis[u]=1;
        for(v=0;v<n;v++)
```

```

        if(g[u][v] && !vis[v] && g[u][v]<key[v]){
            key[v]=g[u][v]; par[v]=u;
        }
    }
    printf("MST Edges:\n");
    for(i=1;i<n;i++){
        printf("%d - %d : %d\n",par[i],i,g[par[i]][i]);
        cost+=g[par[i]][i];
    }
    printf("Total Cost = %d\n",cost);
}

```

Output:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\8_prims.exe
2 0 3 8
0 3 0 7
6 8 7 0
MST Edges:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
Total Cost = 11
❖ PS C:\MinGW\bin>

```

8B – Kruskal's Algorithm

Aim:

To find the Minimum Spanning Tree (MST) using Kruskal's algorithm (Union-Find). Time

Complexity: $O(E \log E)$.

Program (C):

```
#include<stdio.h>

int parent[20];

int find(int x)
{
    while(parent[x])
        x=parent[x];
    return x;
}

int main()
{
    int n,i,j,min,u,v,a,b;
    int edges=0,cost=0;

    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int g[20][20];
    printf("Enter cost matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
```

```

        {
            scanf("%d",&g[i][j]);
            if(i==j || g[i][j]==0)
                g[i][j]=999;
        }
    }
    printf("Edges in MST:\n");
    while(edges<n-1)
    {
        min=999;
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                if(g[i][j]<min)
                {
                    min=g[i][j];
                    a=u=i;
                    b=v=j;
                }
            }
        }
        u=find(u);
        v=find(v);
        if(u!=v)

```



```

    {
        parent[v]=u;
        printf("%d -> %d = %d\n",a,b,min);
        cost+=min;
        edges++;
    }

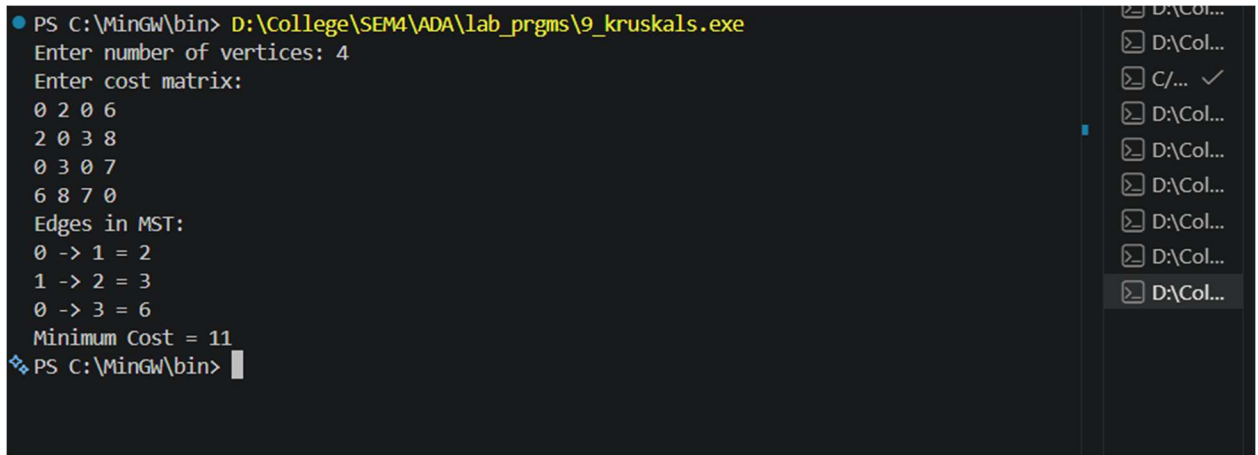
    g[a][b]=g[b][a]=999;
}

printf("Minimum Cost = %d",cost);

return 0;
}

```

Output:



```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\9_kruskals.exe
Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 7
6 8 7 0
Edges in MST:
0 -> 1 = 2
1 -> 2 = 3
0 -> 3 = 6
Minimum Cost = 11
PS C:\MinGW\bin>

```

Lab Program 9:

Experiment Title: Implement Fractional Knapsack using Greedy technique.

Aim:

To solve the Fractional Knapsack Problem using the Greedy approach by selecting items based on profit-to-weight ratio. Time Complexity: $O(N \log N)$.

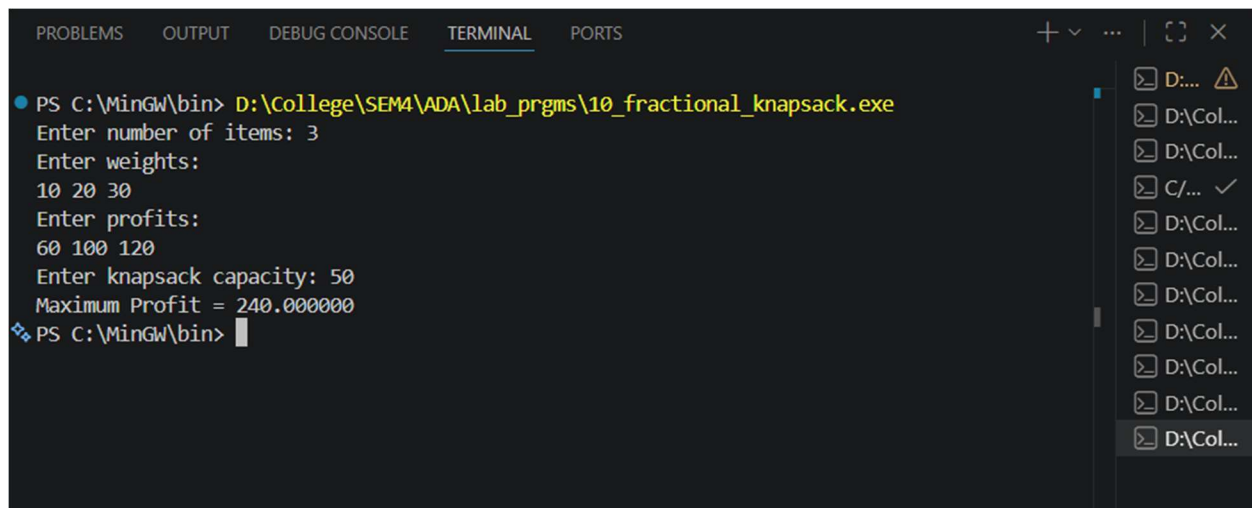
Program (C):

```
#include<stdio.h>

int main()
{
    int n,i,j;
    float wt[20],p[20],r[20],cap,tp=0,t;
    printf("Enter number of items: ");
    scanf("%d",&n);
    printf("Enter weights:\n");
    for(i=0;i<n;i++) scanf("%f",&wt[i]);
    printf("Enter profits:\n");
    for(i=0;i<n;i++) scanf("%f",&p[i]);
    printf("Enter knapsack capacity: ");
    scanf("%f",&cap);
    for(i=0;i<n;i++) r[i]=p[i]/wt[i];
    // Sort by profit/weight ratio descending
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(r[i]<r[j]){
                t=r[i];r[i]=r[j];r[j]=t;
                t=wt[i];wt[i]=wt[j];wt[j]=t;
                t=p[i];p[i]=p[j];p[j]=t;
            }
    for(i=0;i<n;i++){
```

```
        if(wt[i]<=cap){ tp+=p[i]; cap-=wt[i]; }  
        else{ tp+=r[i]*cap; break; }  
    }  
    printf("Maximum Profit = %f",tp);  
}
```

Output:



The screenshot shows a terminal window with a dark background. The title bar includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The terminal content shows the execution of a program named 10_fractional_knapsack.exe. The user enters the number of items as 3, weights as 10 20 30, profits as 60 100 120, and knapsack capacity as 50. The program outputs a maximum profit of 240.000000. The right sidebar shows a file explorer with several files, including D:\Col...

```
PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\10_fractional_knapsack.exe  
Enter number of items: 3  
Enter weights:  
10 20 30  
Enter profits:  
60 100 120  
Enter knapsack capacity: 50  
Maximum Profit = 240.000000  
PS C:\MinGW\bin>
```

Lab Program 10:

Experiment Title: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Aim:

To find the single-source shortest paths from a given vertex to all other vertices using Dijkstra's greedy algorithm. Time Complexity: $O(V^2)$.

Program (C):

```
#include<stdio.h>

int main()
{
    int n,i,j,u,v,min,s;

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    int a[20][20],d[20],vis[20]={0};

    printf("Enter cost matrix:\n");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);

            if(i!=j && a[i][j]==0)

                a[i][j]=999;
        }
    }
}
```

```

printf("Enter source vertex: ");
scanf("%d",&s);
for(i=0;i<n;i++)
    d[i]=a[s][i];
d[s]=0;
vis[s]=1;
for(i=0;i<n-1;i++)
{
    min=999;
    for(j=0;j<n;j++)
    {
        if(!vis[j] && d[j]<min)
        {
            min=d[j];
            u=j;
        }
    }
    vis[u]=1;
    for(v=0;v<n;v++)
    {
        if(!vis[v] &&
            d[u]!=999 &&
            a[u][v]!=999 &&
            d[u]+a[u][v] < d[v])
        {

```

```

        d[v]=d[u]+a[u][v];
    }
}
}

printf("Shortest distances:\n");

for(i=0;i<n;i++)

    printf("%d -> %d = %d\n",s,i,d[i]);

return 0;
}

```

Output:



```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\11_dijkstra.exe
Enter number of vertices: 5
Enter cost matrix:
0 10 0 0 5
0 0 1 0 2
0 0 0 4 0
7 0 6 0 0
0 3 9 2 0
Enter source vertex: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 8
0 -> 2 = 9
0 -> 3 = 7
0 -> 4 = 5
PS C:\MinGW\bin>

```

Lab Program 11:

Experiment Title: Implement "N-Queens Problem" using Backtracking.

Aim:

To place N queens on an N×N chessboard such that no two queens attack each other, using the Backtracking technique. Time Complexity: O(N!).

Program (C):

```
#include<stdio.h>
#include<stdlib.h>
int x[20],n;
int place(int k,int i)
{
    int j;
    for(j=1;j<k;j++)
        if(x[j]==i || abs(x[j]-i)==abs(j-k)) return 0;
    return 1;
}
void queen(int k)
{
    int i,j,y=1;
    for(i=1;i<=n;i++)
    {
        if(place(k,i))
        {
            x[k]=i;
            if(k==n){
                for(j=1;j<=n;j++){
                    printf("Row-%d \t Column-%d\n",y,x[j]);
                    y++;
                }
            }
        }
    }
}
```

```

        printf("\n");
    }
    else queen(k+1);
}
}
}
int main()
{
    printf("Enter value of N: ");
    scanf("%d",&n);
    printf("Solutions:\n");
    queen(1);
}

```

Output:



```

PS C:\MinGW\bin> D:\College\SEM4\ADA\lab_prgms\12_n_queens.exe
Enter value of N: 4
Solutions:
Row-1    Column-2
Row-2    Column-4
Row-3    Column-1
Row-4    Column-3

Row-1    Column-3
Row-2    Column-1
Row-3    Column-4
Row-4    Column-2

PS C:\MinGW\bin>

```


Leetcode: 207

Soln:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* inDegree = (int*)calloc(numCourses, sizeof(int));
    int* graphColSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(numCourses * sizeof(int));
    }
    // Build graph and in-degree array
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];
        graph[prereq][graphColSize[prereq]++] = course;
        inDegree[course]++;
    }
    // Queue for BFS
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    // Add courses with in-degree 0
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i;
        }
    }
}
```

```

int count = 0;

while (front < rear) {

    int current = queue[front++];

    count++;

    for (int i = 0; i < graphColSize[current]; i++) {

        int neighbor = graph[current][i];

        inDegree[neighbor]--;

        if (inDegree[neighbor] == 0) {

            queue[rear++] = neighbor;

        }

    }

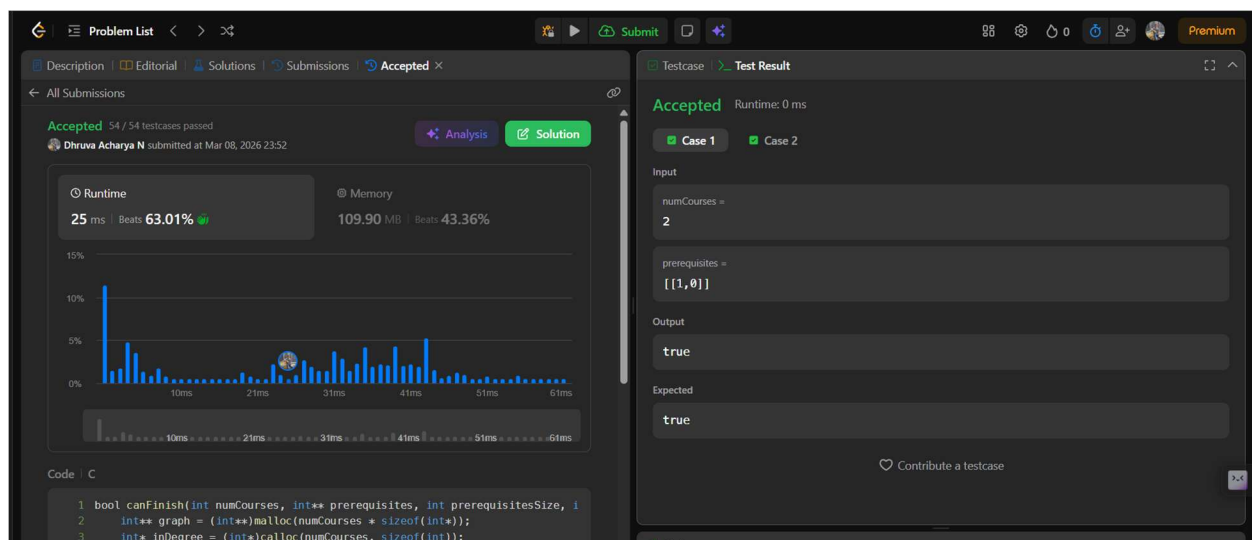
}

return count == numCourses;

}

```

Output:



Leetcode: 287

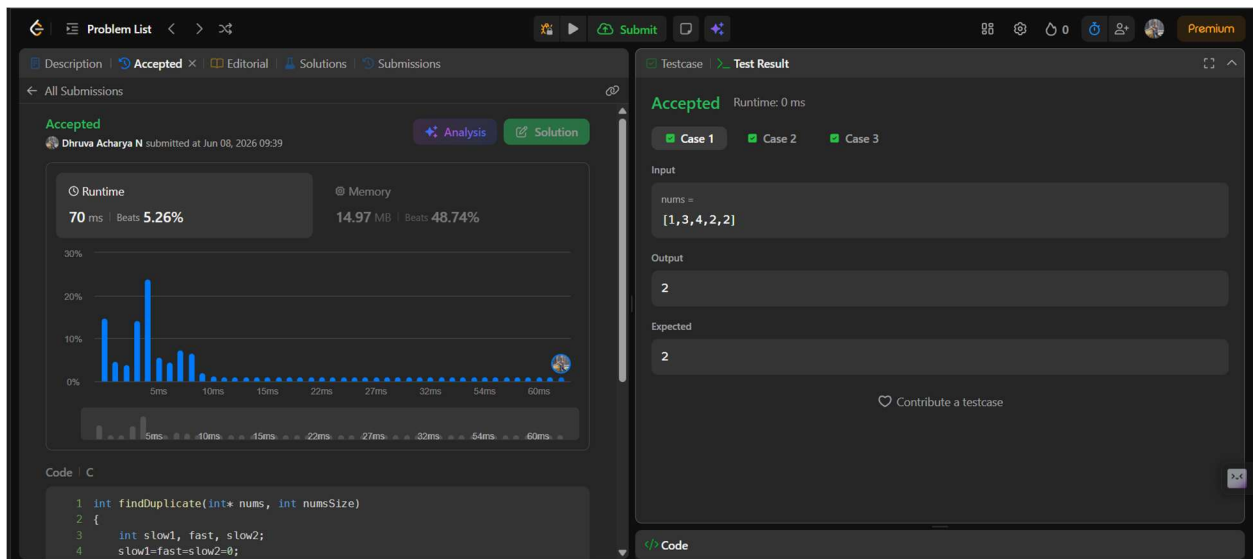
Soln:

```
int findDuplicate(int* nums, int numsSize)
```

```
{  
    int slow1, fast, slow2;  
    slow1=fast=slow2=0;  
  
    while(true)  
    {  
        slow1=nums[slow1];  
        fast=nums[nums[fast]];  
        if(slow1==fast)  
        {  
            break;  
        }  
    }  
}
```

```
while(true)  
{  
    slow1=nums[slow1];  
    slow2=nums[slow2];  
    if(slow1==slow2)  
    {  
        return slow2;  
    }  
}  
}
```

Output:



Leetcode: 801

Soln:

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {  
    int a = 0, b = 1;  
    for (int i = 1; i < nums1Size; ++i) {  
        int x = a, y = b;  
        if (nums1[i - 1] >= nums1[i] || nums2[i - 1] >= nums2[i]) {  
            a = y; b = x + 1;  
        } else {  
            b = y + 1;  
            if (nums1[i - 1] < nums2[i] && nums2[i - 1] < nums1[i]){  
                a = fmin(a, y); b = fmin(b, x + 1);  
            }  
        }  
    }  
    return fmin(a, b);  
}
```

Output:

